

BCP Route Filtering:
 while configuring route map, we can configure using access-list 2 ways
 if you write deny in access-list then you should permit in route-map.
 permit — deny in route-map.

```

access-list 20 permit 20.0.0.0 0.255.255.255
route-map CCIE deny <number>
  match ip address 20
route-map CCIE permit 20
router top 10
  neighbor 1.1.1.2 route-map CCIE in
  " 2.2.2.1 " " "
  
```

TFTP (Trivial File Transfer Protocol)

(1) It is a type of protocol used to transfer file of networking device like routers switches etc.

(2) It is an application layer protocol

→ It uses UDP port no 69

→ File is always transferred or received in the form of 512 bytes.

Meaning of Trivial: Simple & basic

→ No encryption → No hashing → No security → No authentication

where we use TFTP:

→ During backup and restore of configuration file of network device

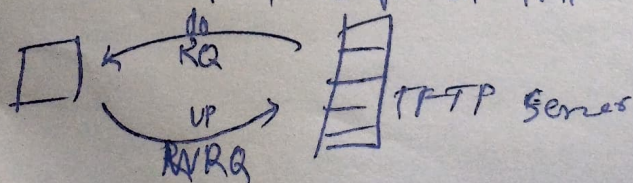
→ Firmware or IOS installation or upgradation

How TFTP works: (It generates ack for received packet)

but it doesn't base on UDP or TCP

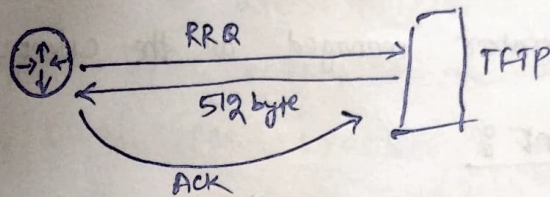
Read Request (RQ) — client download the file from TFTP

Write Request (WRQ) — client upload the file on TFTP



Sequence 1: when Client want to download file from TFTP

- (i) client send read request to server
- (ii) server reply by sending 512 bytes of data block
- (iii) After receiving client send back the ack of each 512 bytes block



Sequence 2: when client want to upload file on TFTP

- (i) Client send write request to server
- (ii) Server receives
- (iii) Client starts sending 512 bytes of data to server
- (iv) Server sends ack for each block after receiving.



SNMP (Simple Network Management Protocol)

(i) → Designed by: IETF (Internet Engineering Task Force)

- It is an application layer protocol
- It uses UDP port no 161 and 162.
- Uses UDP protocol no 17
- There are 3 version used in SNMP
- It works based on server-client model

Purpose of SNMP

- Used for monitoring, management and troubleshooting in the network.
- Networking devices like routers, switches and etc

Components of SNMP:

- SNMP Server
- SNMP Agent

SNMP Server/Manager:

- It is hardware server on which monitoring software is running.
- Like: Solarwind Orion, PRTG, Zabbix and etc
- Server collect all the details from SNMP client and display it in GUI.
- Network Administrator managed all the client from server

SNMP Agent/client:

- Networking devices like router and switches, in which SNMP is configured, performs as agents.

SNMP Messages:

- There are 3 messages used in SNMP.

Get message: Sent by Manager to agent

Set message: sent by manager to agent

Trap/inform message: sent by agent to server

(V1)

(V2 or V3)

Get: Sent by the server to collect the info of agent like:
uptime, interface status & etc.

Set message: Sent by the server to agent to apply some configuration on client

Trap/Inform message: from client to server to update the alert like high CPU, high bandwidth, etc.

SNMP versions (3)

SNMP v1: Less secure, Less complex

v2: Upgrades from v1

v3: more secure, more complex like authentication and encryption like MD5, SHA

SSH (Secure Shell)

→ It is one of the most essential tools any one working with remote server specially if you are managing ^{your} own infrastructure or deploying application.

→ SSH is a protocol for connecting to a remote computers securely.
→ This connection is established over an encrypted channel which means all data sent between ^{your} machine and server is protected from leaves dropping and interception.

→ SSH ^{primarily} mainly used for:

- Remote command-line access
- File transfers
- Tunneling traffic

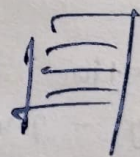
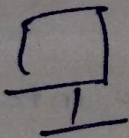
→ It's mainly used by administrators ~~such as~~ ^{for} tasks such as:

- Server Maintenance
- Application deployment
- Troubleshooting

How do ^{you} authenticate to SSH?

→ Password Authentication
(u & p)

→ SSH Key Authentication
(public and private)



• client generates ~~pair of~~ SSH keys on the local machine, the public key and the private key. Private key stored securely in client machine while the public key shared freely with the server.

• When client attempt to connect to the server, the server checks the ^{if the} client has the public key. If the public key matches, the server sends encrypted challenge a random string to the client. Client uses its private key to decrypt the challenge sent by the server, the decrypted message is sent back to the server as

a response, the server uses the public key to verify the response. If the response matches the expected value the server authenticates the client. Once authenticated a secure encrypted SSH session is established between the client and server. Hence the user can run commands or transfer files securely. This is how SSH works.

FTP (File Transfer Protocol)

- FTP is a set of rules and conventions that govern how files are transferred between a client and a server on the internet.
- FTP protocol operates at the application layer of the OSI model. This is the seventh and topmost layer responsible for user interface and application level functions.
- FTP uses a client server model where the server acts as a depository for files and then client can establish connections to perform various operations such as downloading, uploading, deleting and listing directories.
- FTP operates exclusively over TCP and establishes two types of connections between client and server:
 - a control connection
 - a data connection
- The control connection is established by the client on port 21 and is used to send essential information such as user id and password and commands like changing directories or requesting files.
- After receiving necessary control information the server initiates a data connection to the client on port 20. This data connection is responsible for transferring the actual files from the server to the client.
- FTP uses various commands to interact with the server including:
 - get - to download files
 - put - upload files
 - ls - list directory content
 - cd - navigate to another directory

→ The problem with the FTP is that it's not secure because it sends data including usernames and passwords in plain text over the internet making it vulnerable to interception by hackers and unauthorized access.

→ This lack of ^{encryption means that} sensitive information can be easily exposed during file transfers.

→ To ensure the security of your data it's essential to consider more secure alternatives like SFTP ^{or SSH} and FTPS which encrypt your information keeping it safe from eavesdropping.

→ SFTP - SSH File Transfer Protocol :

• keeps your files safe by using encryption during transfers.

• It works on one port usually port 22

→ FTPS - FTP Secure

• It adds security to ~~regular security~~ regular FTP using SSL or TLS encryption.

• It can use two ports " one for control and another for data

→ Explicit FTPS

• Control channel (Command Port): TCP 21

• Data channel (passive mode): Random high TCP ports (usually in the range 1024-65535, configurable on the server)

• Encryption: Starts as plain FTP, then upgraded to TLS/SSL via auth TLS command.

→ Implicit FTPS (less common/legacy)

• Control channel: TCP 990

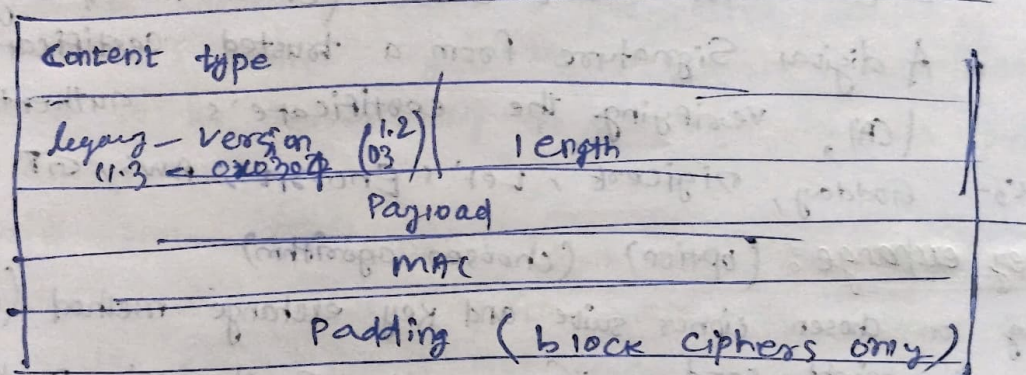
• Data channel: TCP 990

• Encryption SSL/TLS is negotiated immediately upon connections.

TLS

- (i) According to RFC 8446, TLS is application protocol independent; higher-level protocols can layer on top of TLS transparently.
- Think of TLS as a secure tunnel. It doesn't care what kind of data you send through it.
 - Whether you are sending website data (HTTP), emails (SMTP, IMAP), or files (FTP), TLS can secure it.
 - It just wraps your existing communication in secure "envelope" without changing how your application works. So, your web browser (using HTTP) or email client (using SMTP) doesn't need to be redesigned to use TLS; it just uses TLS as a secure

(ii)



message-1 (Client - Hello)

Message 1 in the TLS handshake is the ClientHello, sent by the client to start the secure connection. It includes the TLS version, a random number (client random) - later used for key generation, supported cipher suites (like AES-GCM), and extensions such as SNI for the hostname and supported elliptic curves for key exchange.

message-2 (Server Hello)

The Server will send this message in response to a clientHello message to proceed with the handshake if it is able to negotiate an acceptable set of handshake parameters based on the clientHello.

NOTE:

(12)

(Hello retry request)

Sometimes server says "retry, again", then client send the same Hello message by changing the same contents in the message.

When server

Hello Retry request: The server will send this message in response to a client Hello message if it is able to find an acceptable set of parameters but the client Hello doesn't contains sufficient info to proceed with handshake.

3. Server Certificate:

The server sends its digital SSL/TLS Certificate to the client.

This certificate contains:-

- The Server's public key (X.509 (RSA, ECC, etc.))
- Information about Server (Domain name, Organization, etc.)
- A digital Signature from a trusted Certificate Authority (CA), verifying the certificate's authenticity.

Ex:- Godaddy, Digicert, Let's Encrypt, AWS, etc.

4. Server key exchange: (option) (chosen algorithm)

Depending on chosen cipher suite and key exchange method (e.g. Diffie-Hellman), the server might send a "Server Key Exchange" message containing parameters needed for key exchange.

When key is needed? (NO)

- RSA Key Exchange: The client encrypts the pre-master secret directly with server's public key (from its certificate), so no extra key exchange is needed.

Ephemeral DH/ECDH: (yes)

Since the server generates a temporary key pair, it must send its public key to the client.

(TLS 1.3, the server's key share is sent in the ServerKeyExchange extension)

(6) Certificate Request (optional):

The certificate request message is an optional message. If the server requires client authentication (e.g., for accessing sensitive internal systems), it sends a "Certificate Request" message asking the client for its Certificate.

(6) Server hello Done:

The server sends a "serverHelloDone" message, indicating that it has finished its initial part of the handshake.

(7) Client certificates (optional)

If the server requested a client certificate, the client sends its own certificate.

(8) Client Key Exchange:

This is where the actual shared secret is established. The method depends on the key exchange algorithm chosen:

• RSA: The client generates a "pre-master secret," encrypts it with server's public key (from the server's certificate), and sends it to the server. Only the server, with its private key, can decrypt this.

• Diffie - Hellman (DH/ECDH): Both the client and server exchange public parameters, which allows them to independently calculate the same "pre-master secret" without ever sending it over the network. This is preferred as it provides forward secrecy, meaning even if the server's private key is compromised later, past session data can't be decrypted.

(9) client (change cipher spec)

The client sends a "changeCipherSpec" message, indicating that all subsequent messages from the client will be encrypted using the newly derived session keys.

(10) client (Finish)

The client sends a "Finished" message, which is a hash of all the handshake messages exchanged so far, encrypted with the new session keys. This allows the server to verify that the handshake wasn't tampered with and that the client correctly derived the session keys.

(1) Server (change cipher spec)

The server sends its own "ChangeCipherSpec" message.

(2) Server (Finish)

The server sends its "finished" message, similarly encrypted and containing a hash of the handshake messages.

TLS (1.3)

TLS 1.3 significantly simplifies the handshake process, often reducing it to just one "round trip" (1-RTT) in many cases, or even zero round trips (0-RTT) for session resumption.

1. Client Hello:

- The client sends a ClientHello message, similar to TLS 1.2, but with a few key differences:

- It proactively offers key shares for potential key exchange methods it supports (e.g., Diffie-Hellman parameters). This is a big change, as it means the client can send key material before the server has even chosen a cipher suite.

- It indicates its TLS version ~~preference~~ preference using the supported versions extension. (The legacy-version field in the ClientHello is usually set to TLS 1.2 for backward compatibility reasons).

- It might include "pre-shared-key" (PSK) information if it's attempting 0-RTT session resumption.

2. Server Hello, Server Certificate, Finished (in one message)

- The server receives the ClientHello. If it can process one of the client's offered key shares, it generates its own key share and immediately calculates the shared secret.

- The server responds with a "ServerHello" message, which now combines several previous steps:

- Selected TLS version (TLS 1.3)

- Selected key share (the server's public key share for the chosen key exchange algorithm).

- The server's: encrypted certificate message (encrypted with keys derived from the shared secret, providing greater privacy for certificate information).
 - The server's CertificateVerify message (a signature over the handshake messages to prove ownership of the private key corresponding to the certificate).
 - The server's Finished message (encrypted and authenticated, proving the server correctly derived the session keys).
3. Client Finished:
- The client receives the ServerHello and verifies the server's certificate and signature.
 - It calculates the shared secret based on the server's key share.
 - The client sends its encrypted Finished message, confirming its part of the handshake.

TCP/IP

The TCP/IP model shows the more common terms and layers used when people talk about TCP/IP today. The bottom layer focuses on how to transmit bits over each individual link. The data-link layer focuses on sending data ^{over} one type of physical link: for instance, networks use different data-link protocols for Ethernet LANs versus wireless LANs. The network layer focuses on delivering data over the entire path from the original sending computer to final destination computer. And the top two layers focus more on the applications that need to send and receive data.

TELNET (Teletype Network)

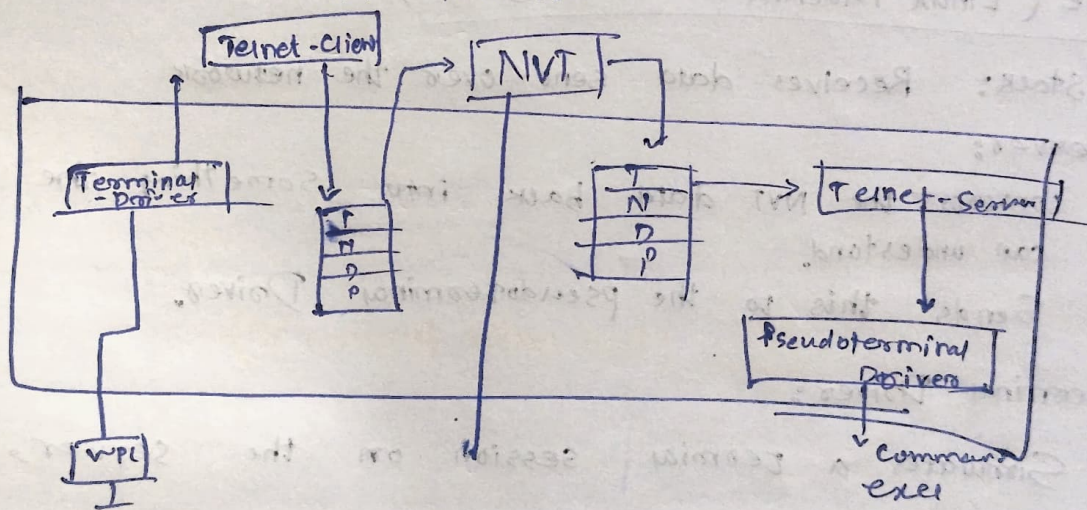
- Telnet stands Teletype Network
- It was initially developed in the year 1969 for ARPANET.
- TCP (port 23)
- Client ↔ Server

What can we do with Telnet

- Server administration
- Configuration of Network devices
- Starting programs remotely
- Editing documents etc.
- Checking, if ports are open (nmap)

command:

telnet <ip> <port-no>



Adv's

- Fast
- Remote communication
- bidirect
- platform-independent

Dis adv's

- Insecure (plain-text)
- Security flaws

→ Telnet allows a user to remotely access another device (server) via Command-line interface (CLI).

→ ~~Windows machine~~

Windows machine - Telnet Client:

- Terminal (CLI/Console): where the user types commands
- Terminal Drivers Translates user input into data Telnet can use.
- Telnet-Client:
 - Sends user Commands over the network
 - Converts the user input to a standard format called NVT (Network Virtual Terminal).

• TCP/IP Stack (Transport, Network, Data Link, Physical)

• moves the Telnet data over the network to the server

Receives

Right side (Linux Machine - Telnet Server):

• TCP/IP Stack: Receives data sent over the network

• Telnet-server:

→ converts the NVT data back into something the server can understand.

→ Sends this to the pseudoterminal Drivers.

• Pseudoterminal Drivers:

→ Simulates a terminal session on the server, like a local user

→ Sends output back to Telnet Server → Client → User screen

NVT (Network Virtual Terminal):

- Acts as a translator between different OS terminal formats.
- Standardizes communication between any client and server regardless of OS.

Both client and server use 4 OSI layers here:

- | | |
|--------------|----------------------|
| 1. Physical | 3. Network |
| 2. Data Link | 4. Transport (TCP) - |

1. we type a command in windows Terminal

2. Tinet Client converts it to NVT format.

3. Sent over the TCP/IP network stack.

4. ~~Sent over~~ Tinet Server receives it, decode NVT.

5. Pseudo-terminal runs the command as if it was typed locally

6. Output is sent back to the client and shown on your screen.